PowerShell for the Data & ML Engineer

A practical tutorial for Python / cron / AWS / Mongo / MySQL users moving onto Windows

Abstract

This tutorial teaches the PowerShell you actually need as a data engineer or machine-learning engineer on Windows: navigating the filesystem, transforming data with the object pipeline, parsing CSV and JSON, calling REST APIs, managing environment variables for AWS / MongoDB / MySQL credentials, and running Python with `venv` and `uv`. Each concept is introduced through a worked example, with a side-by-side comparison to the bash or Python equivalent you already know. PowerShell 7+ is assumed throughout (the cross-platform `pwsh`); 5.1-only quirks are flagged when they matter.

Contents

1 What PowerShell Is (and Isn't)

The single most important thing to internalize before you write any PowerShell: **PowerShell pipes objects, not text.**

In bash, every command emits a stream of bytes, and the next command in the pipeline parses those bytes back into a structure. If you want the size of every file over 1 MB, you run `ls -l`, then `awk` to chop columns, then `sort -n`, and a typo in the column index silently breaks everything. In PowerShell, `Get-ChildItem` returns real `FileInfo` objects with properties like `Length`, `Name`, `LastWriteTime`, and the next stage of the pipeline filters and sorts by those properties directly. There is no parsing step, and no formatting choices to break.

```
# All files over 1 MB in the current directory, biggest first.
Get-ChildItem |
    Where-Object Length -gt 1MB |
    Sort-Object Length -Descending |
    Select-Object Name, Length
```

```
# Roughly the same in bash -- and notice how much
# more fragile the column parsing is.
ls -la | awk '$5 > 1048576 {print $9, $5}' | sort -k2 -nr
```

The Verb-Noun convention

Every cmdlet (PowerShell's word for a command) is named `Verb-Noun`: `Get-ChildItem`, `Set-Location`, `New-Item`, `Remove-Item`, `Import-Csv`, `Invoke-RestMethod`. The verbs come from a small, deliberate vocabulary (`Get`, `Set`, `New`, `Remove`, `Add`, `Test`, `Invoke`, `Out`, `ConvertTo`, `ConvertFrom`, ...). Once you know roughly what you want to do, you can often guess the cmdlet name.

Cmdlets, aliases, and discovery

Most Unix command names work in PowerShell as *aliases* for real cmdlets. `ls` is an alias for `Get-ChildItem`; `cd` is an alias for `Set-Location`; `cat` is `Get-Content`; `rm` is `Remove-Item`. Aliases are fine in the interactive shell, but **use the full cmdlet name in scripts** so the script is portable and self-documenting.

Three cmdlets you will use constantly to learn the rest:

```
Get-Command *csv*           # find every cmdlet whose name contains "csv"
Get-Help Import-Csv -Full   # the full help page (parameters, examples)
Get-Item .\data.csv | Get-Member   # list every property and method of an object
```

`Get-Member` is the secret weapon: *pipe anything to it* and PowerShell tells you exactly which properties and methods that object exposes. This is how you discover, for example, that the result of `Invoke-RestMethod` on a JSON endpoint is already a real object with dot-accessible properties.

Compare with what you know

In Python, you call `dir(obj)` or `help(obj)` to explore an object's interface. `Get-Member` is the same idea, with the bonus that it works inline at the end of a pipeline.

2 Getting Around: Files and Directories

The cmdlets in this section replace the daily-driver Unix commands. They all accept paths with spaces if you quote them, and they all take `-Recurse` and `-Force` flags where you would expect.

```
Get-Location                              # pwd
Set-Location C:\Users\shiru\Desktop       # cd
Get-ChildItem                              # ls
Get-ChildItem -Recurse -Filter *.parquet  # find . -name "*.parquet"
Get-ChildItem -Force                       # ls -a (include hidden files)

New-Item -ItemType Directory data\raw     # mkdir data\raw
New-Item -ItemType Directory -Force a\b\c # mkdir -p a\b\c
New-Item -ItemType File notes.txt         # touch notes.txt

Copy-Item .\data.csv .\backup\data.csv
Copy-Item -Recurse .\src .\src_backup     # cp -r
Move-Item .\old_name.csv .\new_name.csv   # mv (also rename)
Remove-Item .\tmp.csv                     # rm
Remove-Item -Recurse -Force .\.venv       # rm -rf

Test-Path .\data.csv                       # [ -e data.csv ]; returns
    $true/$false
Resolve-Path .\data.csv                    # absolute path
```

Paths and quoting

PowerShell paths use backslash on Windows (`C:\Users\shiru\data.csv`) but forward slash works too. Surround paths containing spaces with double quotes:

```
Get-Content "C:\Users\shiru\My Documents\notes.txt"
```

The backtick (`) is the escape character (*not* backslash), and the dollar sign (`$`) is special inside double quotes (it interpolates). Inside single quotes, nothing is special:

```
Write-Output "Today is $env:USERNAME"     # interpolates -> "Today is shiru"
Write-Output 'Today is $env:USERNAME'     # literal      -> "Today is
    $env:USERNAME"
Write-Output "Price is `$5"               # backtick-escape -> "Price is $5"
```

Safety: -WhatIf and -Confirm

Every destructive cmdlet (`Remove-Item`, `Stop-Process`, `Set-Content` on an existing file, ...) accepts `-WhatIf` to *print what would happen without doing it*, and `-Confirm` to prompt before each action. Get in the habit of running anything you're not sure about with `-WhatIf` first:

```
Remove-Item -Recurse -Force .\.venv -WhatIf
# What if: Performing the operation "Remove Directory" on target ".venv".
```

Compare with what you know

Unix has nothing equivalent. The closest is running a `find` command before a `find -delete` to inspect the matches. With `-WhatIf` every destructive cmdlet has dry-run built in.

3 The Object Pipeline

This is the section that pays for the whole tutorial. If you only learn five PowerShell cmdlets, learn these:

Cmdlet	What it does
`Where-Object`	filter (`WHERE`, `df.query`, `filter()`)
`Select-Object`	project columns / take N (`SELECT`, `df[[...]]`)
`ForEach-Object`	map / transform each item (`map`, `apply`)
`Sort-Object`	sort by a property (`ORDER BY`, `df.sort_values`)
`Group-Object`	group by a key (`GROUP BY`, `df.groupby`)
`Measure-Object`	count / sum / avg / min / max (`aggregate`)

The objects flow left-to-right; the variable `$_` refers to the current object inside a { ... } script block.

Worked example: log file analysis

Suppose you have a directory of `*.log` files and want to know which file has the most lines mentioning `ERROR`.

```
Get-ChildItem -Filter *.log |
    ForEach-Object {
        [PSCustomObject]@{
            File   = $_.Name
            Errors = (Select-String -Path $_.FullName -Pattern 'ERROR').Count
        }
    } |
    Sort-Object Errors -Descending |
    Select-Object -First 5
```

This is the entire pipeline, end to end: enumerate files, build a small object per file with `File` and `Errors` properties, sort by error count, take the top 5.

Block form vs. property form

`Where-Object` has two syntaxes. They do the same thing; the *property form* is shorter and works for simple comparisons.

```
# Property form (simple comparison).
Get-ChildItem | Where-Object Length -gt 1MB

# Block form (any expression).
Get-ChildItem | Where-Object { $_.Length -gt 1MB -and $_.Name -like '*.csv' }
```

Operators are spelled out (not symbolic) because `<` and `>` are reserved for redirection. The full set you'll use:

Operator	Meaning
`-eq`, `-ne`	equal, not equal
`-lt`, `-le`	less than, less or equal
`-gt`, `-ge`	greater than, greater or equal
`-like`	wildcard match (`*`, `?`) – case insensitive
`-clike`	wildcard, case sensitive (prefix `c` on any op makes it case-sensitive)
`-match`	regex match (populates `$Matches`)
`-notmatch`	negated regex
`-in`, `-notin`	membership: `$x -in @(1,2,3)`
`-contains`, `-notcontains`	reverse membership: collection `-contains $x`
`-and`, `-or`, `-not`, `-xor`	boolean combinators
`-is`, `-isnot`	type check: `$x -is [string]`

These operators broadcast naturally over collections, so `@(1,2,3,4,5) -gt 2` returns `@(3,4,5)` – the same trick lets you do quick set filters without an explicit `Where-Object` call.

Group, count, summarize

Counting events by category is one line:

```
# How many files per extension in this directory tree?
Get-ChildItem -Recurse -File |
    Group-Object Extension |
    Sort-Object Count -Descending |
    Select-Object Count, Name
```

`Measure-Object` does numeric aggregation:

```
Get-ChildItem -Recurse -File |
    Measure-Object Length -Sum -Average -Maximum
# Count : 1842
# Average : 28734.21
```

```
# Sum     : 52928711
# Maximum : 14523904
```

Parallel execution (PowerShell 7+)

`ForEach-Object` accepts a `-Parallel` switch in PowerShell 7+ that runs the script block on multiple threads – ideal for I/O-bound work like hitting hundreds of API endpoints or reading many files at once.

```
# Fetch metadata for 100 URLs in parallel, 10 at a time.
$urls = Get-Content .\urls.txt
$urls | ForEach-Object -Parallel {
    $r = Invoke-RestMethod -Uri $_
    [PSCustomObject]@{ url = $_; status = $r.status }
} -ThrottleLimit 10 |
    Export-Csv -NoTypeInformation -Path .\statuses.csv -Encoding utf8
```

Two things to know:

- Inside `-Parallel`, the variable `$_` still refers to the current item, but *outside-scope variables are not visible* – pass them in with `$using:varname`.
- Order is not preserved. Use it for embarrassingly parallel work; for sequential ETL where each step depends on the previous, stick with plain `ForEach-Object`.

Compare with what you know

The PowerShell pipeline is so close to a pandas method chain that the mapping is almost mechanical:

```
df[df.size > 1e6].sort_values('size', ascending=False).head(5)
```

becomes

```
Get-ChildItem | Where-Object Length -gt 1MB | Sort-Object Length -Descending | Select-Object -F
```

4 Variables and Data Types

Variables

Variable names start with `$`. Assignment uses `=`; types are inferred but you can cast.

```
$name = 'shiru'
$count = 42
$ratio = 0.95
$today = Get-Date

# Casting:
$n = [int]"7"           # 7
$x = [double]"3.14"     # 3.14
```

```
$d = [datetime]"2026-01-15"
```

PowerShell variables are not scoped to a block by default (unlike Python's function-scoped locals); they live in the enclosing scope. For scripts, this is rarely an issue, but be aware of it when you write loops.

Arrays

The `@(...)` syntax builds an array; you can also build one implicitly by listing items separated by commas. Indexing is zero-based and uses square brackets, exactly like Python.

```
$nums = @(1, 2, 3, 4, 5)
$nums[0]        # 1
$nums[-1]       # 5  (negative indexing works)
$nums[1..3]     # 2, 3, 4  (range slicing)
$nums.Length    # 5
$nums += 6      # append (creates a NEW array; arrays are immutable)

# Iterate:
foreach ($n in $nums) { Write-Output ($n * 2) }

# Or use the pipeline:
$nums | ForEach-Object { $_ * 2 }
```

Gotcha

The `+=` on arrays is *not* cheap. It creates a brand-new array each time. For large collections, build a `[System.Collections.Generic.List[object]]` instead, or use the pipeline to emit items.

Hashtables (dicts)

Hashtables are PowerShell's dictionary type.

```
$user = @{
    name = 'shiru'
    role = 'Data Engineer'
    languages = @('Python', 'SQL', 'PowerShell')
}

$user.name              # 'shiru'
$user['role']           # 'Data Engineer' (both syntaxes work)
$user.email = 'shirui7352136@gmail.com'   # add a key
$user.ContainsKey('name')                  # $true
$user.Keys
$user.Values
```

Compare with what you know

Python dict literal `{"name": "shiru", "role": "DE"}` is PowerShell's `@{name='shiru'; role='DE'}`. Note the semicolons (or newlines) between entries – not commas.

Custom objects

Cmdlets in a pipeline want objects, not bare hashtables. Promote a hashtable to an object with `[PSCustomObject]`:

```
$row = [PSCustomObject]@{
    user_id = 42
    score   = 0.91
    label   = 'churn'
}
$row | Get-Member        # see the properties
$row | Export-Csv -NoTypeInformation -Path .\out.csv
```

This is how you build CSV-shaped data on the fly and pipe it into the data-wrangling cmdlets you'll meet in Section 7.

Booleans and $null

`$true`, `$false`, and `$null` are the three constants you need. The comparison operators short-circuit (`-and`, `-or`, `-not`). Anything zero-length, zero-valued, or `$null` is falsy in conditions:

```
if ($name) { Write-Output "got a name" }
if ($null -eq $row.email) { Write-Output "email is missing" }
```

5 Strings and Formatting

PowerShell strings come in four useful forms. Pick the right one to avoid the most common Windows footgun (mangled escapes).

```
$name = 'shiru'

"Hello $name"              # double-quoted: variables interpolate
'Hello $name'              # single-quoted: literal, no interpolation
"Sum: $($a + $b)"          # subexpression form for complex inserts
"Path: $env:USERPROFILE"   # env: drive interpolates too

# Here-strings preserve newlines and ignore most quoting issues:
$body = @"
{
    "user": "$name",
    "count": 42
}
"@                         # closing "@ MUST be at column 0
```

```
# Single-quoted here-string -- literal, no interpolation, ideal for
# regex patterns or JSON templates with literal $ in them:
$pattern = @'
^\$[0-9]+\.[0-9]{2}$
'@
```

Gotcha

Inside a here-string, the closing `"@` or `'@` must be at the start of the line, with no leading whitespace. Indenting it is a parse error.

Formatting numbers and dates

The `-f` operator is the printf-style formatter. It uses .NET format specifiers.

```
"{0:N2}"   -f 1234.5678              # "1,234.57"  (2-decimal, with separators)
"{0:P1}"   -f 0.952                  # "95.2 %"     (percent)
"{0:F4}"   -f (1/3)                  # "0.3333"     (fixed-point)
"{0:yyyy-MM-dd}" -f (Get-Date)       # "2026-05-25"
"{0,-10} {1,5}" -f 'shiru', 42       # "shiru          42"  (padded columns)
```

Regex on strings

```
'AWS-prod-eu-west-1' -match '^AWS-(prod|stage)-([a-z0-9-]+)$'
# $true -- and the captures land in $Matches:
$Matches[1]   # 'prod'
$Matches[2]   # 'eu-west-1'

'foo,bar,baz' -split ','                # @('foo','bar','baz')
'foo bar baz' -replace '\s+', '_'       # 'foo_bar_baz'

# Multiline regex on a file:
Select-String -Path .\app.log -Pattern '^\d{4}-\d{2}-\d{2}.*ERROR' |
    Select-Object LineNumber, Line
```

Compare with what you know

`-replace` is regex by default. To do a literal replacement, use the `.Replace()` string method: `'a.b.c'.Replace('.', '/')`.

6 File I/O – Plain Text

`Get-Content` reads, `Set-Content` writes (overwrite), `Add-Content` appends.

```
# Read whole file into a single string vs. array of lines.
$lines = Get-Content .\app.log               # array, one element per line
$blob  = Get-Content .\app.log -Raw          # single string with embedded newlines
```

```powershell
# Write / overwrite a file.
Set-Content -Path .\notes.txt -Value "First line"
Set-Content -Path .\notes.txt -Value @('First', 'Second', 'Third')    # array ->
    multi-line

# Append.
Add-Content -Path .\notes.txt -Value "Another line"

# Stream a large file line-by-line without loading it all into memory.
Get-Content .\huge.log -ReadCount 0 | ForEach-Object {
    if ($_ -match 'ERROR') { $_ }
}
```

Encoding

The single most common Windows footgun: `Set-Content` and `Out-File` write UTF-8 *with BOM* by default in PowerShell 5.1 and UTF-8 *without BOM* in PowerShell 7+. If you're writing data that will be read by a non-Windows tool (Python's `open(..., encoding='utf-8')`, AWS S3, jq), be explicit:

```powershell
Set-Content -Path .\out.txt -Value $text -Encoding utf8       # UTF-8 without BOM
    on PS7
Set-Content -Path .\out.txt -Value $text -Encoding utf8NoBOM  # explicit, PS7 only
Set-Content -Path .\out.txt -Value $text -Encoding ascii      # ASCII
```

Gotcha

`Out-File` has different default encoding from `Set-Content` historically. In any code that crosses tool boundaries, **always pass** `-Encoding utf8` explicitly.

7 Data Wrangling – CSV

This is where PowerShell starts to feel genuinely good for data work. `Import-Csv` reads a CSV and yields **one object per row**, with column names as properties. No manual splitting, no quoting bugs, no off-by-one indexing.

Example file `users.csv`:

```
user_id,name,role,active,score
1,Alice,Data Engineer,true,0.91
2,Bob,ML Engineer,true,0.87
3,Carol,Analyst,false,0.62
4,Dan,Data Engineer,true,0.78
```

Read, filter, project, write

```powershell
$users = Import-Csv .\users.csv
```

```powershell
# Filter rows where active is "true" and score >= 0.8.
$top = $users |
    Where-Object { $_.active -eq 'true' -and [double]$_.score -ge 0.8 } |
    Select-Object name, role, score

$top | Format-Table          # pretty print
$top | Export-Csv -NoTypeInformation -Path .\top.csv -Encoding utf8
```

Two things to notice:

1. Every column comes back as a *string*, even `score`. You have to cast with `[double]` (or `[int]`, `[bool]`, `[datetime]`) before numeric comparisons.
2. `-NoTypeInformation` is critical when writing – otherwise `Export-Csv` prepends an annoying `#TYPE` comment line that confuses every non-PowerShell consumer.

Group by, aggregate

```powershell
# Average score per role.
Import-Csv .\users.csv |
    Group-Object role |
    ForEach-Object {
        [PSCustomObject]@{
            role      = $_.Name
            n         = $_.Count
            avg_score = ($_.Group | Measure-Object {[double]$_.score}
    -Average).Average
        }
    } |
    Sort-Object avg_score -Descending |
    Format-Table
```

Joining two CSVs

PowerShell has no SQL-style `JOIN`, but the hashtable-as-index trick gets you the same answer with no extra dependencies. Suppose `orders.csv` has columns `order_id,user_id,amount`.

```powershell
# Build a lookup: user_id -> user object.
$userById = @{}
Import-Csv .\users.csv | ForEach-Object { $userById[$_.user_id] = $_ }

# Stream orders, attach the user's name and role to each order.
Import-Csv .\orders.csv |
    ForEach-Object {
        $u = $userById[$_.user_id]
        [PSCustomObject]@{
            order_id = [int]$_.order_id
            user_id  = [int]$_.user_id
            name     = $u.name
            role     = $u.role
```

```
            amount   = [double]$_.amount
        }
    } |
    Export-Csv -NoTypeInformation -Path .\orders_joined.csv -Encoding utf8
```

This is O(n+m) and streams orders one at a time – so it handles million-row order tables just fine as long as `users.csv` fits in RAM. For a many-to-many join, the lookup value becomes a list and you call `.Add()` on it.

Different delimiters and encodings

```
Import-Csv .\eu_users.csv -Delimiter ';'            # European-style CSV
Import-Csv .\data.tsv     -Delimiter "`t"           # tab-separated; backtick-t is
    the tab escape
Import-Csv .\latin1.csv   -Encoding latin1
```

When to drop into Python

`Import-Csv` loads the whole file into memory as objects – fine for tens of thousands of rows, slow for millions. For anything in `pandas` territory (joins, time-series resampling, vectorized math), let PowerShell handle the orchestration and shell out to a short Python script for the heavy work:

```
# orchestrate from PowerShell, compute in pandas
python -m my_pipeline.transform --input .\users.csv --output .\users_enriched.csv
$enriched = Import-Csv .\users_enriched.csv
```

Compare with what you know

`Import-Csv` is to `pandas.read_csv` what `csv.DictReader` is to `pandas.read_csv`: row-oriented, object-per-row, all values as strings unless you cast. Use it for small-to-medium files and orchestration; reach for pandas when you need columnar speed.

8 Data Wrangling – JSON and HTTP

Parsing and emitting JSON

```
# Parse a JSON file into a real object.
$config = Get-Content .\config.json -Raw | ConvertFrom-Json

# Navigate with dot notation, just like in Python.
$config.database.host
$config.database.replicas[0].port

# Modify and write back. -Depth defaults to 2, which is too shallow for
# almost any real config. Always pass -Depth explicitly when writing.
$config.database.host = 'mongo-prod-1'
$config | ConvertTo-Json -Depth 10 | Set-Content -Path .\config.json -Encoding utf8
```

Gotcha

`ConvertTo-Json`'s default `-Depth` is 2. Anything nested deeper than that gets stringified as `"System.Collections.Hashtable"` without warning. **Always pass `-Depth 10`** (or higher) for any non-trivial structure.

Hitting a REST API

`Invoke-RestMethod` is one cmdlet that does `requests.get(...).json()` in one step: it sends the HTTP request *and* parses the JSON response into a navigable object.

```
# Simple GET.
$resp = Invoke-RestMethod -Uri 'https://api.github.com/repos/python/cpython'
$resp.stargazers_count
$resp.license.spdx_id

# GET with headers (e.g. bearer token from an env var).
$headers = @{ Authorization = "Bearer $env:GITHUB_TOKEN" }
$prs = Invoke-RestMethod -Uri
    'https://api.github.com/repos/python/cpython/pulls?state=open' `
                          -Headers $headers
$prs | Select-Object number, title, user | Format-Table

# POST with a JSON body.
$body = @{ name = 'shiru'; role = 'DE' } | ConvertTo-Json -Depth 5
Invoke-RestMethod -Uri 'https://httpbin.org/post' `
                  -Method Post `
                  -ContentType 'application/json' `
                  -Body $body
```

The backtick at end-of-line is PowerShell's line-continuation character (the equivalent of bash's trailing backslash). Use it sparingly; most of the time you can just break inside parentheses or at a pipe symbol.

Paginated APIs

A practical pattern – consume a paginated REST endpoint and stream results into a single array.

```
$all = @()
$page = 1
do {
    $items = Invoke-RestMethod `
        -Uri "https://api.example.com/items?page=$page&per_page=100" `
        -Headers @{ Authorization = "Bearer $env:API_TOKEN" }
    $all += $items
    $page++
} while ($items.Count -eq 100)

$all | ConvertTo-Json -Depth 10 |
```

```
    Set-Content -Path .\all_items.json -Encoding utf8
```

Compare with what you know

The PowerShell version is shorter than the Python equivalent because `Invoke-RestMethod` bakes in JSON parsing and HTTP error handling. The trade-off: no async, no session pooling, fewer auth strategies. For high-throughput ingestion, use Python's `httpx` or `aiohttp`.

Error handling and retries

`Invoke-RestMethod` throws a terminating error on non-2xx responses. Wrap calls in `try / catch` and inspect `$_.Exception.Response.StatusCode` to decide whether to retry, skip, or fail loudly.

```
function Invoke-WithRetry {
    param(
        [string]$Uri,
        [hashtable]$Headers = @{},
        [int]$MaxAttempts = 5
    )
    for ($attempt = 1; $attempt -le $MaxAttempts; $attempt++) {
        try {
            return Invoke-RestMethod -Uri $Uri -Headers $Headers
        } catch {
            $code = $_.Exception.Response.StatusCode.value__
            if ($code -in 429, 500, 502, 503, 504) {
                $backoff = [Math]::Pow(2, $attempt)
                Write-Warning "HTTP $code on attempt $attempt; sleeping $backoff s"
                Start-Sleep -Seconds $backoff
            } else {
                throw   # 4xx other than 429 -> don't retry
            }
        }
    }
    throw "Gave up after $MaxAttempts attempts: $Uri"
}

$data = Invoke-WithRetry -Uri 'https://api.example.com/items' `
                         -Headers @{ Authorization = "Bearer $env:API_TOKEN" }
```

This is the same exponential-backoff pattern you'd write around `requests.get(...)` in Python, just in PowerShell's syntax.

9 Environment Variables and PATH

This is where you wire up credentials and config for AWS, MongoDB, MySQL, OpenAI, etc.

Reading and writing in the current session

```
$env:AWS_PROFILE                  # read (returns $null if unset)
$env:AWS_PROFILE = 'production'   # set for THIS process only
Remove-Item Env:\AWS_PROFILE      # unset for this process

# Inspect everything:
Get-ChildItem Env:
Get-ChildItem Env: | Where-Object Name -like 'AWS_*'

# Inspect PATH (it's just a string, semicolon-separated on Windows):
$env:Path -split ';'
```

Persisting across sessions

`$env:FOO = '...'` only affects the current process. To make a variable stick across PowerShell restarts and reboots, write it to the user or machine environment:

```
# User-scoped (just you, all your shells): persists across sessions
[Environment]::SetEnvironmentVariable('AWS_PROFILE', 'production', 'User')

# Read it back from the persisted store (not from the current process):
[Environment]::GetEnvironmentVariable('AWS_PROFILE', 'User')

# Machine-scoped (every user on this PC): requires admin shell
[Environment]::SetEnvironmentVariable('CUDA_HOME', 'C:\CUDA\v12.5', 'Machine')

# Prepending to PATH (user-scoped):
$path = [Environment]::GetEnvironmentVariable('Path', 'User')
[Environment]::SetEnvironmentVariable('Path', 'C:\tools;' + $path, 'User')
```

Gotcha

Setting a User or Machine env var *does not affect the current PowerShell session.* You must open a new shell, or refresh `$env:Path` manually with `$env:Path = [Environment]::GetEnvironmentVariable('Path','Machine') + ';' + [Environment]::GetEnvironmentVariable('Path','User')`.

A practical credentials pattern

Don't paste credentials into scripts. Set them once at the user level, read them in the script via `$env:NAME`.

```
# One-time setup at the user level:
[Environment]::SetEnvironmentVariable('AWS_ACCESS_KEY_ID',     'AKIA...', 'User')
[Environment]::SetEnvironmentVariable('AWS_SECRET_ACCESS_KEY', '...',     'User')
[Environment]::SetEnvironmentVariable('AWS_DEFAULT_REGION',    'us-east-1', 'User')
[Environment]::SetEnvironmentVariable('MONGO_URI',
    'mongodb+srv://...', 'User')
[Environment]::SetEnvironmentVariable('MYSQL_HOST',            'db.internal',
    'User')
```

```
# In every later script, just read:
$client = New-MongoClient -Uri $env:MONGO_URI
```

Compare with what you know

Unix shells use `export FOO=bar` (process-scoped, with the `.bashrc` / `.zshrc` dance for persistence). PowerShell gives you both knobs explicitly: `$env:FOO = ...` for the process, `[Environment]::SetEnvironmentVariable(...)` for the registry-backed persistent store.

10 Python on Windows PowerShell

Where is Python?

`Get-Command` is your `which`:

```
Get-Command python
# CommandType  Name        Source
# Application  python.exe  C:\Users\shiru\miniconda3\python.exe

Get-Command python -All     # every python.exe on PATH, in order
```

If you have multiple Python installations (conda, the Microsoft Store Python, the official installer...), the first one on `$env:Path` wins. `Get-Command python -All` shows them all so you can reorder PATH or pin a specific path in a script.

The `py` launcher (optional)

The official Python installer for Windows ships a `py.exe` launcher that picks the right Python version from a single command. It is *not* installed by Conda or Microsoft Store, so it may not be on your machine. If it is, you can do:

```
py -0p             # list every Python interpreter the launcher knows about
py -3.12 script.py # run with Python 3.12 specifically
py -3 -m venv .venv
```

If you don't have `py`, just use `python` directly – nothing in this tutorial requires the launcher.

Running scripts

```
python script.py                    # equivalent to bash: python3 script.py
python -m pytest                    # run a module as a script
python -m pip install requests      # always prefer `python -m pip` over bare
    `pip`
python -c "import sys; print(sys.executable)"
```

The `python -m` form is portable across all OSes and avoids any ambiguity about which `pip` or `pytest` is on PATH.

11 Virtual Environments with venv

Create and activate

```
python -m venv .venv                 # create the venv in .\.venv
.\.venv\Scripts\Activate.ps1         # activate it (note: \Scripts\, not \bin\)
# Prompt becomes (.venv) PS C:\...>
python -m pip install requests pandas boto3 pymongo mysql-connector-python
deactivate                           # leaves the venv
```

The execution policy wall

The first time you run `Activate.ps1` on a Windows machine, you'll get this error:

```
.\.venv\Scripts\Activate.ps1 : File C:\Users\shiru\Desktop\powershell\.venv\
Scripts\Activate.ps1 cannot be loaded because running scripts is disabled on
this system. ...
```

This is PowerShell's execution policy refusing to run unsigned local scripts. Fix it once, at the user scope (**never machine scope**), then forget about it:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

`RemoteSigned` means: local scripts run freely; scripts *downloaded from the internet* must be signed or unblocked. This is the right balance for a developer machine, and it only affects your user account.

Compare with what you know

There is no Unix equivalent. The execution policy is a Windows-only trust mechanism. `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` is the universally-recommended one-time fix.

Worked example: per-project venv

```
# From a fresh project directory:
Set-Location C:\Users\shiru\projects\churn-model
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
# work, work, work...
deactivate
```

To re-enter the venv tomorrow, the only command you need is `.\.venv\Scripts\Activate.ps1`.

12 uv on PowerShell

uv is the fast Python package and project manager you already use. On Windows it installs without admin rights and integrates with PowerShell exactly the same way as on Linux.

Install

The official installer is a single PowerShell command:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

irm is the alias for Invoke-RestMethod; iex is Invoke-Expression. Together they fetch the installer script and run it. After install, open a new PowerShell tab so uv appears on PATH, and verify:

```
uv --version
```

Daily commands

```
# Start a new project (creates pyproject.toml, .venv, and uv.lock):
uv init churn-model
Set-Location churn-model

# Add dependencies (updates pyproject.toml and uv.lock automatically):
uv add pandas boto3 pymongo "mysql-connector-python>=8.3"
uv add --dev pytest ruff

# Run a Python script through uv (no manual activation needed):
uv run python -m churn_model.train --input data\users.csv

# Or run a one-off command in the project env:
uv run pytest
uv run ruff check .

# Sync the env to match uv.lock exactly (use this on CI / a fresh clone):
uv sync

# Lock without installing (useful in CI):
uv lock
```

When you do want to activate

uv run usually means you never have to activate the venv. But if you want a shell with the venv on PATH (e.g. for interactive work), activate it like any other venv:

```
.\.venv\Scripts\Activate.ps1
```

Working with `requirements.txt`

If you have a legacy `requirements.txt`, `uv` has a fast pip-compatible interface:

```
uv venv                                          # create .venv
uv pip install -r requirements.txt              # fast pip install into .venv
uv pip compile requirements.in -o requirements.txt   # resolve & pin
uv pip sync requirements.txt                     # make .venv match requirements.txt
    EXACTLY
```

Compare with what you know

The mental model: `uv` is to `pip + venv + pip-tools + virtualenv` what `cargo` is to Rust's loose tooling. `uv run` replaces `source .venv/bin/activate && python ...` as a one-liner.

13 Putting It Together: A Mini ETL Walkthrough

This section ties everything together. Goal: read a CSV of users, hit a REST API to enrich each row with a profile lookup, run a small Python transformation in a `uv`-managed venv to compute a per-user score, write the result as JSON, log progress to a daily log file.

Project layout

```
C:\projects\enrich\
    pyproject.toml          # managed by uv
    uv.lock
    .venv\                   # created by `uv venv`
    data\
        users.csv           # input
    enrich.ps1              # the orchestrator (PowerShell)
    score.py                # the transformation (Python)
    logs\
        enrich-2026-05-25.log
```

The Python transformation (`score.py`)

```
# score.py -- reads enriched rows from stdin (JSON lines),
# emits scored rows to stdout (JSON lines).
import json, sys

for line in sys.stdin:
    rec = json.loads(line)
    base = float(rec.get("score", 0))
    boost = 0.1 if rec.get("verified") else 0.0
    rec["final_score"] = round(base + boost, 4)
    sys.stdout.write(json.dumps(rec) + "\n")
```

The orchestrator (`enrich.ps1`)

```
# enrich.ps1
param(
    [string]$Input  = '.\data\users.csv',
    [string]$Output = '.\data\users_scored.json'
)

$ErrorActionPreference = 'Stop'

# Set up a daily log file.
$logDir = '.\logs'
if (-not (Test-Path $logDir)) { New-Item -ItemType Directory $logDir | Out-Null }
$logFile = Join-Path $logDir ("enrich-{0:yyyy-MM-dd}.log" -f (Get-Date))

function Log {
    param([string]$Msg)
    $line = "{0:yyyy-MM-dd HH:mm:ss} {1}" -f (Get-Date), $Msg
    Add-Content -Path $logFile -Value $line
    Write-Output $line
}

Log "Reading $Input"
$users = Import-Csv $Input

Log "Enriching $($users.Count) users via API"
$enriched = foreach ($u in $users) {
    try {
        $profile = Invoke-RestMethod -Uri
    "https://api.example.com/users/$($u.user_id)" `
                                    -Headers @{ Authorization = "Bearer
    $env:API_TOKEN" }
        [PSCustomObject]@{
            user_id  = [int]$u.user_id
            name     = $u.name
            role     = $u.role
            score    = [double]$u.score
            verified = $profile.verified
        }
    } catch {
        Log "WARN: failed to enrich user_id=$($u.user_id): $_"
    }
}

Log "Scoring via Python in uv-managed venv"
$jsonLines = $enriched | ForEach-Object { $_ | ConvertTo-Json -Compress -Depth 10 }
$scored = $jsonLines | uv run python score.py | ForEach-Object {
    $_ | ConvertFrom-Json
}

Log "Writing $($scored.Count) records to $Output"
```

```
$scored | ConvertTo-Json -Depth 10 | Set-Content -Path $Output -Encoding utf8

Log "Done"
```

Run it

```
$env:API_TOKEN = 'sk-...'
.\enrich.ps1 -Input .\data\users.csv -Output .\data\users_scored.json
```

What this example shows, in one place:

- Parameters with defaults (`param(...)`).
- Error policy (`$ErrorActionPreference = 'Stop'`) so an unhandled error halts the script.
- File-system setup with `Test-Path` + `New-Item`.
- A small reusable function (`Log`).
- CSV input via `Import-Csv`, with typed casting on the way in.
- REST enrichment via `Invoke-RestMethod` with a bearer-token header from `$env`.
- Hand-off to a Python script via `uv run python score.py`, streaming JSON lines through stdin/stdout.
- JSON output with explicit `-Depth` and explicit `-Encoding`.

14 Common Pitfalls

Execution policy. The first `Activate.ps1` run fails on every fresh Windows install. Fix once with `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

Path quoting. Use double quotes around paths with spaces. The escape character is the backtick, not the backslash. To put a literal `$` inside a double-quoted string, prefix it with a backtick: `"`$5"`.

`ConvertTo-Json` **depth.** The default `-Depth` is 2. Nested data silently turns into the string `"System.Collections.Hashtable"`. Always pass `-Depth 10` (or higher) for anything non-trivial.

File encoding. `Out-File` and `Set-Content` have different defaults in PowerShell 5.1 vs. 7. Always pass `-Encoding utf8` when the file will be read by non-Windows tools.

`Import-Csv` **columns are strings.** Cast with `[double]`, `[int]`, `[bool]`, `[datetime]` before any numeric or date comparison.

`$env:` **doesn't persist.** Setting `$env:FOO = 'bar'` only affects the current process. Use `[Environment]::SetEnvironmentVariable(...)` to persist.

Array += is O(n). For large collections, use `[System.Collections.Generic.List[object]]::new()` and call `.Add()`, or just emit items through the pipeline and let it do the right thing.

Here-string closer at column 0. The closing `"@` or `'@` must be at the start of the line, with no leading whitespace. Indenting it is a parse error.

`-WhatIf` before destructive cmdlets. Every `Remove-`, `Stop-`, `Clear-` cmdlet accepts `-WhatIf`. Make it a reflex.

PowerShell 5.1 vs. 7. On a modern Windows install, the default `powershell.exe` is still PowerShell 5.1; the cross-platform `pwsh.exe` is PowerShell 7+. Anything in this tutorial that uses `utf8NoBOM` or `??` or the ternary operator is PowerShell 7 only. Always launch `pwsh`, not `powershell`, for new work.

15 Quick Reference: Bash → PowerShell

The minimal table you need pinned somewhere. PowerShell aliases (`ls`, `cd`, `cat`, ...) work in the interactive shell but use the cmdlet name in scripts.

Bash	PowerShell
`pwd`	`Get-Location`
`cd /path`	`Set-Location C:\path`
`ls`	`Get-ChildItem`
`ls -la`	`Get-ChildItem -Force`
`find . -name "*.csv"`	`Get-ChildItem -Recurse -Filter *.csv`
`cat file`	`Get-Content file`
`head -n 10 file`	`Get-Content file -TotalCount 10`
`tail -n 10 file`	`Get-Content file -Tail 10`
`tail -f file`	`Get-Content file -Wait -Tail 0`
`wc -l file`	`(Get-Content file | Measure-Object -Line).Lines`
`grep PATTERN file`	`Select-String -Pattern PATTERN -Path file`
`which python`	`(Get-Command python).Source`
`mkdir dir`	`New-Item -ItemType Directory dir`
`mkdir -p a/b/c`	`New-Item -ItemType Directory -Force a\b\c`
`touch f`	`if (-not (Test-Path f)) { New-Item -ItemType File f }`
`cp src dst`	`Copy-Item src dst`
`cp -r src dst`	`Copy-Item -Recurse src dst`
`mv src dst`	`Move-Item src dst`
`rm f`	`Remove-Item f`
`rm -rf dir`	`Remove-Item -Recurse -Force dir`
`ln -s tgt link`	`New-Item -ItemType SymbolicLink -Path link -Target tgt`
`echo $VAR`	`$env:VAR`
`export VAR=x`	`$env:VAR = 'x'` (process-scoped)
`export VAR=x` (persistent)	`[Environment]::SetEnvironmentVariable('VAR','x','User'`
`cmd > out`	`cmd | Set-Content out`
`cmd >> out`	`cmd | Add-Content out`
`cmd1 && cmd2`	`cmd1 && cmd2` (PowerShell 7+)
`cmd1 || cmd2`	`cmd1 || cmd2` (PowerShell 7+)
`a=$(cmd)`	`$a = (cmd)`
`curl URL | jq .`	`Invoke-RestMethod URL`
`2>/dev/null`	`2>$null`
`for f in *.csv; do ...done`	`Get-ChildItem *.csv | ForEach-Object { ...}`

One last thing: when you're stuck, the answer almost always starts with `Get-Command *word*` (find the cmdlet), `Get-Help NAME -Examples` (see how to use it), and `Get-Member` (see what properties an object has). PowerShell rewards discovery in a way bash never did.